

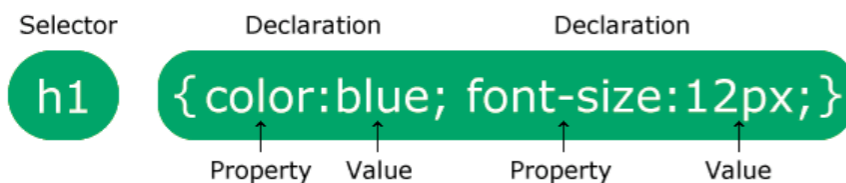
CSS:

CSS is the language we use to style a Web page.

- CSS stands for Cascading Style Sheets
- CSS describes how HTML elements are to be displayed on screen, paper, or in other media
- CSS saves a lot of work. It can control the layout of multiple web pages all at once
- External stylesheets are stored in CSS files

CSS Syntax

A CSS rule consists of a selector and a declaration block:



The selector points to the HTML element you want to style.

The declaration block contains one or more declarations separated by semicolons.

Each declaration includes a CSS property name and a value, separated by a colon.

CSS Selectors

CSS selectors are used to "find" (or select) the HTML elements you want to style.

All CSS Simple Selectors

Selector	Example	Example description
<u>#id</u>	#firstname	Selects the element with id="firstname"
<u>.class</u>	.intro	Selects all elements with class="intro"
<u>*</u> _	*	Selects all elements
<u>element</u>	p	Selects all <p> elements
<u>element,element,...</u>	div, p	Selects all <div> elements and all <p> elements

CSS Types :

There are three ways of inserting a style sheet:

- External CSS
- Internal CSS
- Inline CSS

External CSS

With an external style sheet, you can change the look of an entire website by changing just one file!

Each HTML page must include a reference to the external style sheet file inside the <link> element, inside the head section.

```

<> data.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <link rel="stylesheet" href="mystyle.css" />
5    </head>
6    <body>
7      <h1>This is a heading</h1>
8      <p>This is a paragraph.</p>
9    </body>
10   </html>
11

```

Internal CSS:

An internal style sheet may be used if one single HTML page has a unique style.

The internal style is defined inside the <style> element, inside the head section.

```

<> data.html > html > body > h1
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <style>
5        body {
6          background-color: linen;
7        }
8
9        h1 {
10         color: maroon;
11         margin-left: 40px;
12       }
13     </style>
14   </head>
15   <body>
16     <h1>heading</h1>
17     <p>This is a paragraph.</p>
18   </body>
19   </html>
20

```

Inline CSS:

An inline style may be used to apply a unique style for a single element.

To use inline styles, add the style attribute to the relevant element. The style attribute can contain any CSS property.

```

data.html > ...
1  <!DOCTYPE html>
2  <html>
3    <body>
4      <h1 style="color: blue; text-align: center">This is a heading</h1>
5      <p style="color: red">This is a paragraph.</p>
6    </body>
7  </html>
8

```

```

new-js > one.html > html > body > h2.inter-blue
1  <html>
2    <head>
3      <title>CSS Page</title>
4      <link rel="stylesheet" href="style.css" />
5
6      <style>
7        .inter {
8          color: brown;
9        }
10
11        .inter-blue {
12          color: green;
13        }
14      </style>
15    </head>
16
17    <body>
18      <!--Inline styling example-->
19
20      <h1 style="color: red">Red styling</h1>
21      <h2 style="color: blue">Blue styling</h2>
22
23      <!--Inline styling example-->
24
25      <!--Internal styling example-->
26
27      <h1 class="inter">Brown styling Internal CSS</h1>
28      <h2 class="inter-blue">Green styling Internal CSS</h2>
29
30      <!--Internal styling example-->
31    </body>
32  </html>
33

```

CSS Comments:

```
# style.css > ...
1  /* This is a single-line comment */
2  p {
3    color: red;
4  }
5
6  /* This is
7   a multi-line
8   comment */
9
10 p {
11   color: red;
12 }
13
```

CSS Colors:

In CSS, colors are specified by using a predefined color name, or with a RGB, HEX, HSL

RGB Value

An RGB color value represents RED, GREEN, and BLUE light sources.

In CSS, a color can be specified as an RGB value, using this formula:

rgb(red, green, blue)

Each parameter (red, green, and blue) defines the intensity of the color between 0 and 255.

For example, `rgb(255, 0, 0)` is displayed as red, because red is set to its highest value (255) and the others are set to 0.

To display black, set all color parameters to 0, like this: `rgb(0, 0, 0)`.

To display white, set all color parameters to 255, like this: `rgb(255, 255, 255)`.

Experiment by mixing the RGB values below:

`rgb(255, 99, 71)`

RGBA Value

RGBA color values are an extension of RGB color values with an alpha channel - which specifies the opacity for a color.

An RGBA color value is specified with:

`rgba(red, green, blue, alpha)`

The alpha parameter is a number between 0.0 (fully transparent) and 1.0 (not transparent at all):

Experiment by mixing the RGBA values below:

`rgba(255, 99, 71, 0.4)`

HEX Value

A hexadecimal color is specified with: #RRGGBB.

In CSS, a color can be specified using a hexadecimal value in the form:

`#rrggbb`

Where rr (red), gg (green) and bb (blue) are hexadecimal values between 00 and ff (same as decimal 0-255).

For example, #ff0000 is displayed as red, because red is set to its highest value (ff) and the others are set to the lowest value (00).

To display black, set all values to 00, like this: #000000.

To display white, set all values to ff, like this: #ffffff.

Experiment by mixing the HEX values below:

```
> data.html > ...
1  <!DOCTYPE html>
2  <html>
3    <body>
4      <h1>Specify colors using HEX values</h1>
5
6      <h2 style="background-color:  #ff0000">#ff0000</h2>
7      <h2 style="background-color:  #0000ff">#0000ff</h2>
8      <h2 style="background-color:  #3cb371">#3cb371</h2>
9      <h2 style="background-color:  #ee82ee">#ee82ee</h2>
10     <h2 style="background-color:  #ffa500">#ffa500</h2>
11     <h2 style="background-color:  #6a5acd">#6a5acd</h2>
12   </body>
13 </html>
14
```

HSL Value

HSL stands for hue, saturation, and lightness.

In CSS, a color can be specified using hue, saturation, and lightness (HSL) in the form:

hsl(hue, saturation, lightness)

Hue is a degree on the color wheel from 0 to 360. 0 is red, 120 is green, and 240 is blue.

Saturation is a percentage value. 0% means a shade of gray, and 100% is the full color.

Lightness is also a percentage. 0% is black, 50% is neither light or dark, 100% is white

Experiment by mixing the HSL values below:

```

data.html > ...
1  <!DOCTYPE html>
2  <html>
3    <body>
4      <h1>Specify colors using HSL values</h1>
5
6      <h2 style="background-color: red hsl(0, 100%, 50%)">hsl(0, 100%, 50%)</h2>
7      <h2 style="background-color: blue hsl(240, 100%, 50%)">hsl(240, 100%, 50%)</h2>
8      <h2 style="background-color: green hsl(147, 50%, 47%)">hsl(147, 50%, 47%)</h2>
9      <h2 style="background-color: magenta hsl(300, 76%, 72%)">hsl(300, 76%, 72%)</h2>
10     <h2 style="background-color: orange hsl(39, 100%, 50%)">hsl(39, 100%, 50%)</h2>
11     <h2 style="background-color: purple hsl(248, 53%, 58%)">hsl(248, 53%, 58%)</h2>
12   </body>
13 </html>
14

```

CSS Backgrounds

CSS background-color

The [background-color](#) property specifies the background color of an element.

```

data.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <style>
5        h1 {
6          background-color: green;
7        }
8
9        div {
10         background-color: lightblue;
11       }
12
13       p {
14         background-color: yellow;
15       }
16     </style>
17   </head>
18   <body>
19     <h1>CSS background-color example!</h1>
20     <div>
21       This is a text inside a div element.
22       <p>This paragraph has its own background color.</p>
23       We are still in the div element.
24     </div>
25   </body>
26 </html>
27

```


The [opacity](#) property specifies the opacity/transparency of an element. It can take a value from 0.0 - 1.0. The lower value, the more transparent:

```
div {  
  background-color: green;  
  opacity: 0.3;  
}
```

CSS background-image

The [background-image](#) property specifies an image to use as the background of an element.

By default, the image is repeated so it covers the entire element.

```
<> data.html > html > head > style > body  
1  <!DOCTYPE html>  
2  <html>  
3    <head>  
4      <style>  
5        body {  
6          background-image: url("bgdesert.jpg"); /* please provide necessary URL here */  
7        }  
8      </style>  
9    </head>  
10   <body>  
11     <h1>Hello World!</h1>  
12     <p>This text is not easy to read on this background image.</p>  
13   </body>  
14 </html>  
15
```

CSS background-repeat

The [background-repeat](#) property sets if/how a background image will be repeated.

By default, a background-image is repeated both vertically and horizontally.

Some images should be repeated only horizontally or vertically, or they will look strange, like this:

The CSS Background Repeat and Position Properties

Property	Description
background-position	Sets the starting position of a background image
background-repeat	Sets how a background image will be repeated

```
<> data.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <style>
5        body {
6          background-image: url("img_tree.png");
7          background-repeat: no-repeat;
8          background-position: right top;
9          margin-right: 200px;
10         }
11      </style>
12    </head>
13    <body>
14      <h1>Hello World!</h1>
15      <p>
16        Here, the background image is only shown once. In addition it is
17        positioned away from the text.
18      </p>
19      <p>
20        In this example we have also added a margin on the right side, so that the
21        background image will not disturb the text.
22      </p>
23    </body>
24  </html>
25
```

CSS background-attachment

The [background-attachment](#) property specifies whether the background image should scroll or be fixed (will not scroll with the rest of the page):

```
body {  
  background-image: url("img_tree.png");  
  background-repeat: no-repeat;  
  background-position: right top;  
  background-attachment: scroll;  
}
```

CSS Borders

The CSS border properties allow you to specify the style, width, and color of an element's border.

I have borders on all sides.

I have a red, bottom border.

I have rounded borders.

I have a blue, left border.

CSS Border Style

The [border-style](#) property specifies what kind of border to display.

The following values are allowed:

- `dotted` - Defines a dotted border
- `dashed` - Defines a dashed border
- `solid` - Defines a solid border
- `double` - Defines a double border
- `groove` - Defines a 3D grooved border. The effect depends on the border-color value

- **ridge** - Defines a 3D ridged border. The effect depends on the border-color value
- **inset** - Defines a 3D inset border. The effect depends on the border-color value
- **outset** - Defines a 3D outset border. The effect depends on the border-color value
- **none** - Defines no border
- **hidden** - Defines a hidden border
-

The [border-style](#) property can have from one to four values (for the top border, right border, bottom border, and the left border).

```

1  <html>
2  <head>
3  <style>
4  {
5  }
6  p.dashed {
7  border-style: dashed;
8  }
9  p.solid {
10 border-style: solid;
11 }
12 p.double {
13 border-style: double;
14 }
15 p.groove {
16 border-style: groove;
17 }
18 p.ridge {
19 border-style: ridge;
20 }
21 p.inset {
22 border-style: inset;
23 }
24 p.outset {
25 border-style: outset;
26 }
27 p.none {
28 border-style: none;
29 }
30 p.hidden {
31 border-style: hidden;
32 }
33 p.mix {
34 border-style: dotted dashed solid double;
35 }
36 }
37 </style>
38 </head>
39 <body>
40 <h2>The border-style Property</h2>
41 <p>This property specifies what kind of border to display:</p>
42
43 <p class="dotted">A dotted border.</p>
44 <p class="dashed">A dashed border.</p>
45 <p class="solid">A solid border.</p>
46 <p class="double">A double border.</p>
47 <p class="groove">A groove border.</p>
48 <p class="ridge">A ridge border.</p>
49 <p class="inset">An inset border.</p>
50 <p class="outset">An outset border.</p>
51 <p class="none">No border.</p>
52 <p class="hidden">A hidden border.</p>
53 <p class="mix">A mixed border.</p>
54 </body>
55 </html>
56

```

CSS Border Width

The [border-width](#) property specifies the width of the four borders.

The width can be set as a specific size (in px, pt, cm, em, etc) or by using one of the three pre-defined values: thin, medium, or thick:

CSS Border Color

The [border-color](#) property is used to set the color of the four borders.

The color can be set by:

- name - specify a color name, like "red"
- HEX - specify a HEX value, like "#ff0000"
- RGB - specify a RGB value, like "rgb(255,0,0)"
- HSL - specify a HSL value, like "hsl(0, 100%, 50%)"
- transparent

Note: If [border-color](#) is not set, it inherits the color of the element.

CSS Border - Individual Sides

From the examples on the previous pages, you have seen that it is possible to specify a different border for each side.

In CSS, there are also properties for specifying each of the borders (top, right, bottom, and left):

- [border-top-style](#)
- [border-right-style](#)
- [border-bottom-style](#)
- [border-left-style](#)

In the example below we use the four properties above to set the style of each border side:

CSS Border - Shorthand Property

Like you saw in the previous page, there are many properties to consider when dealing with borders.

To shorten the code, it is also possible to specify all the individual border properties in one property.

The [border](#) property is a shorthand property for the following individual border properties:

- [border-width](#)
- [border-style](#) (required)
- [border-color](#)

```
p {border: 5px solid red;}
```

CSS Rounded Borders

The [border-radius](#) property is used to add rounded borders to an element:

CSS Margins

The CSS margin properties are used to create space around elements, outside of any defined borders.

Margins define the distance between an element's border and the surrounding elements.

With CSS, you have full control over the margins. CSS has properties for setting the margin for each individual side of an element (top, right, bottom, and left), and a shorthand property for setting all the margin properties in one declaration.

Margin - Individual Sides

CSS has properties for specifying the margin for each side of an element:

- [margin-top](#) - sets the top margin of an element

- [margin-right](#) - sets the right margin of an element
- [margin-bottom](#) - sets the bottom margin of an element
- [margin-left](#) - sets the left margin of an element

All the margin properties can have the following values:

- auto - the browser calculates the margin
- *length* - specifies a margin in px, pt, cm, etc.
- % - specifies a margin in % of the width of the containing element
- inherit - specifies that the margin should be inherited from the parent element

Tip: Negative values are also allowed.

Margin - Shorthand Property

To shorten the code, it is possible to specify all the margin properties in one declaration.

The [margin](#) property is a shorthand property for the following individual margin properties:

- [margin-top](#)
- [margin-right](#)
- [margin-bottom](#)
- [margin-left](#)

Here is how it works:

If the [margin](#) property has four values:

- margin: 25px 50px 75px 100px;
 - top margin is 25px
 - right margin is 50px
 - bottom margin is 75px
 - left margin is 100px

If the [margin](#) property has three values:

- margin: 25px 50px 75px;
 - top margin is 25px
 - right and left margins are 50px
 - bottom margin is 75px

If the [margin](#) property has two values:

- margin: 25px 50px;
 - top and bottom margins are 25px
 - right and left margins are 50px

If the [margin](#) property has one value:

- margin: 25px;
 - all four margins are 25px

```
<> data.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <style>
5  <div {
6      border: 1px solid black;
7      margin: 25px 50px 75px 100px;
8      background-color: lightblue;
9  }
10 </style>
11 </head>
12 <body>
13   <h2>The margin shorthand property - 4 values</h2>
14
15   <div>
16       This div element has a top margin of 25px, a right margin of 50px, a
17       bottom margin of 75px, and a left margin of 100px.
18   </div>
19
20   <hr />
21 </body>
22 </html>
23
```

CSS Padding

The CSS padding properties are used to generate space around an element's content, inside of any defined borders.

With CSS, you have full control over the padding. There are properties for setting the padding for each side of an element (top, right, bottom, and left), and a shorthand property for setting all the padding properties in one declaration.

Padding - Individual Sides

CSS has properties for specifying the padding for each side of an element:

- [padding-top](#) - sets the top padding of an element
- [padding-right](#) - sets the right padding of an element
- [padding-bottom](#) - sets the bottom padding of an element
- [padding-left](#) - sets the left padding of an element

All the padding properties can have the following values:

- *length* - specifies a padding in px, pt, cm, etc.
- % - specifies a padding in % of the width of the containing element
- inherit - specifies that the padding should be inherited from the parent element

Note: Negative values are not allowed.

Padding - Shorthand Property

To shorten the code, it is possible to specify all the padding properties in one declaration.

The [padding](#) property is a shorthand property for the following individual padding properties:

- [padding-top](#)
- [padding-right](#)
- [padding-bottom](#)
- [padding-left](#)

Here is how it works:

If the [padding](#) property has four values:

- padding: 25px 50px 75px 100px;
 - top padding is 25px
 - right padding is 50px
 - bottom padding is 75px
 - left padding is 100px

If the [padding](#) property has three values:

- padding: 25px 50px 75px;
 - top padding is 25px
 - right and left paddings are 50px
 - bottom padding is 75px

If the [padding](#) property has two values:

- padding: 25px 50px;
 - top and bottom paddings are 25px
 - right and left paddings are 50px

If the [padding](#) property has one value:

- padding: 25px;
 - all four paddings are 25px

```

<> data.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <style>
5  <div {
6      border: 1px solid black;
7      padding: 25px;
8      background-color: lightblue;
9  }
10 </style>
11 </head>
12 <body>
13   <h2>The padding shorthand property - 1 value</h2>
14
15   <div>
16       This div element has a top, bottom, left, and right padding of 25px.
17   </div>
18 </body>
19 </html>
20

```

CSS Height, Width and Max-width

The CSS [height](#) and [width](#) properties are used to set the height and width of an element.

The CSS [max-width](#) property is used to set the maximum width of an element.

CSS Set height and width

The [height](#) and [width](#) properties are used to set the height and width of an element.

The height and width do not include padding, borders, or margins. It sets the height and width of the area inside the padding, border, and margin of the element.

CSS height and width Values

The [height](#) and [width](#) properties can have the following values:

- **auto** - This is default. The browser calculates the height and width
- **length** - Defines the height or width in px, cm, em, etc.

- `%` - Defines the height or width in percent of the containing block
- `initial` - Sets the height or width to its default value
- `inherit` - The height or width will be inherited from its parent value

CSS Using max-width

The `max-width` property sets the maximum allowed width of an element. This prevents the width of an element to be larger than the `max-width` property value.

The `max-width` property can have the following values:

- `length` - Defines the maximum width in px, cm, etc.
- `%` - Defines the maximum width in percent of the containing block
- `none` - This is default. Means that there is no maximum width

One problem with the `width` property can occur when the browser window is smaller than the width of the element. The browser then adds a horizontal scrollbar to the page. So, using `max-width` will improve the browser's handling on small windows.

CSS max-width Examples

Drag the browser window to smaller than 600px wide, to see the difference between the two divs below!

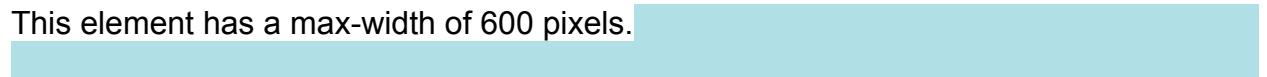
Using width:

This element has a width of 600 pixels.



Using max-width:

This element has a max-width of 600 pixels.



All CSS Dimension Properties

Property	Description
<code>height</code>	Sets the height of an element
<code>max-height</code>	Sets the maximum height of an element
<code>max-width</code>	Sets the maximum width of an element
<code>min-height</code>	Sets the minimum height of an element
<code>min-width</code>	Sets the minimum width of an element
<code>width</code>	Sets the width of an element

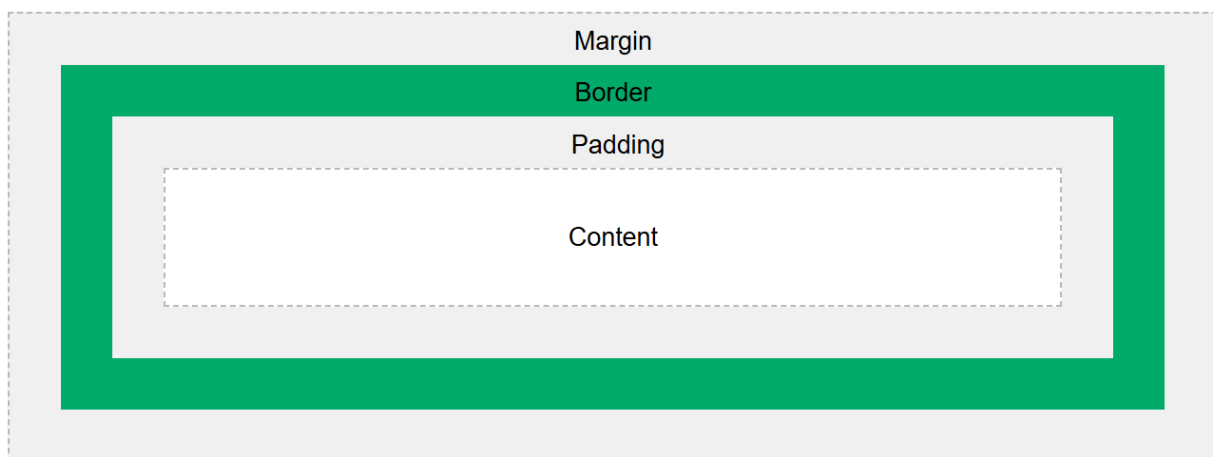
CSS Box Model

In CSS, the term "box model" is used when talking about web design and layout.

The CSS box model is essentially a box that wraps around every HTML element.

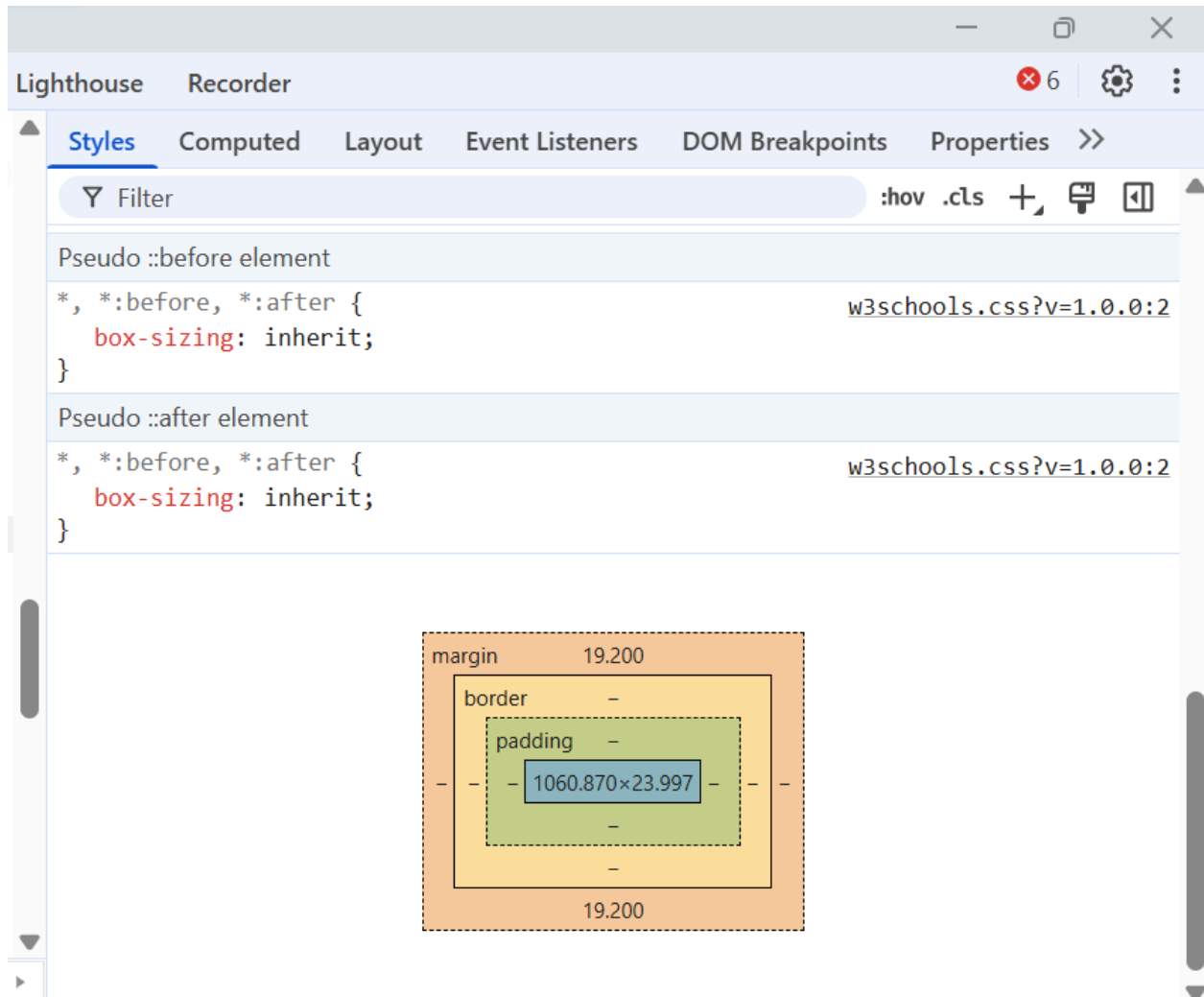
Every box consists of four parts: content, padding, borders and margins.

The image below illustrates the CSS box model:



Explanation of the different parts (from innermost part to outermost part):

- **Content** - The content of the box, where text and images appear
- **Padding** - Clears an area around the content. The padding is transparent
- **Border** - A border that goes around the padding and content
- **Margin** - Clears an area outside the border. The margin is transparent



CSS Outline

An outline is a line that is drawn around elements, OUTSIDE the borders, to make the element "stand out".

Note: Outline differs from [borders](#)! The outline is drawn outside the element's border, and may overlap other content. Also, the outline is NOT a part of the element's dimensions; the element's total width and height is not affected by the width of the outline.

CSS has the following outline properties:

- [outline-style](#) - Specifies the style of the outline
- [outline-color](#) - Specifies the color of the outline
- [outline-width](#) - Specifies the width of the outline
- [outline-offset](#) - Adds space between the outline and the edge/border of an element
- [outline](#) - A shorthand property

```
p.ex3 {outline: 7px solid yellow;}
```

```
> data.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <style>
5        p {
6          padding: 5px;
7        }
8
9        p.ex1 {
10         outline: dashed;
11        }
12        p.ex2 {
13         outline: dotted red;
14        }
15        p.ex3 {
16         outline: 7px solid yellow;
17        }
18        p.ex4 {
19         outline: thick ridge pink;
20        }
21      </style>
22    </head>
23    <body>
24      <h2>The outline shorthand Property</h2>
25
26      <p class="ex1">A dashed outline.</p>
27      <p class="ex2">A dotted red outline.</p>
28      <p class="ex3">A 7px solid yellow outline.</p>
29      <p class="ex4">A thick ridge pink outline.</p>
30    </body>
31  </html>
32
```

```

data.html > html > head > style > p.ex4
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <style>
5    p {
6      border: 1px solid black;
7      padding: 5px;
8    }
9
10   p.ex1 {
11     outline: dashed;
12     border-radius: 8px;
13   }
14
15   p.ex2 {
16     outline: dotted red;
17     border-radius: 5px;
18   }
19
20   p.ex3 {
21     outline: 7px solid yellow;
22     border-radius: 5px;
23   }
24
25   p.ex4 {
26     outline: thick ridge pink;
27     border-radius: 8px;
28   }
29
30   p.ex5 {
31     outline: thick solid green;
32     border-radius: 10px;
33   }
34 </style>
35 </head>
36 <body>
37   <h2>The outline shorthand Property</h2>
38
39   <p class="ex1">A dashed outline.</p>
40   <p class="ex2">A dotted red outline.</p>
41   <p class="ex3">A 7px solid yellow outline.</p>
42   <p class="ex4">A thick ridge pink outline.</p>
43   <p class="ex5">A thick solid green outline.</p>
44 </body>
45 </html>
46

```

- Specifies

CSS Text Color

The [color](#) property is used to set the color of the text. The color is specified by:

- a color name - like "red"
- a HEX value - like "#ff0000"
- an RGB value - like "rgb(255,0,0)"

Look at [CSS Color Values](#) for a complete list of possible color values.

The default text color for a page is defined in the body selector.

CSS Text Alignment and Text Direction

In this chapter you will learn about the following properties:

- [text-align](#)
- [Text-align-last](#)
- [vertical-align](#)

Text Alignment

The [text-align](#) property is used to set the horizontal alignment of a text.

This property can have one of the following values:

- `left` - Aligns the text to the left
- `right` - Aligns the text to the right
- `center` - Centers the text
- `justify` - Stretches the lines so that each line has equal width

The following example shows left, right and center aligned text (left is default if text direction is left-to-right, and right is default if text direction is right-to-left):

Vertical Alignment

The [vertical-align](#) property sets the vertical alignment of an element.

This property can have one of the following values:

- `baseline` - Default value. The element is aligned with the baseline of the parent
- `length/%` - Raises or lower an element by the specified length or percent
- `sub` - The element is aligned with the subscript baseline of the parent
- `super` - The element is aligned with the superscript baseline of the parent
- `top` - The element is aligned with the top of the tallest element on the line

- `text-top` - The element is aligned with the top of the parent element's font
- `middle` - The element is placed in the middle of the parent element
- `bottom` - The element is aligned with the lowest element on the line
- `text-bottom` - The element is aligned with the bottom of the parent element's font

CSS Text Decoration

The CSS [text-decoration](#) property is used to control the appearance of decorative lines on text.

It is a shorthand property for the following individual properties:

- [text-decoration-line](#)
 - [text-decoration-color](#)
 - [text-decoration-style](#)
 - [text-decoration-thickness](#)
-

Add a Decoration Line to Text

The CSS [text-decoration-line](#) property sets the type of decoration line added to the text.

This property can have one or more of the following values:

- `none` - Default value. Displays no decoration line
- `underline` - The decoration line is displayed under the text
- `overline` - The decoration line is displayed over the text
- `line-through` - The decoration line is displayed through the text

Specify a Color for the Decoration Line

The CSS [text-decoration-color](#) property is used to set the color of the decoration line.

Specify a Style for the Decoration Line

The CSS [text-decoration-style](#) property is used to set the style of the decoration line.

This property can have one of the following values:

- `solid` - Default value. Displays the line as a single line
- `double` - Displays the line as a double line
- `dotted` - Displays the line as a dotted line
- `dashed` - Displays the line as a dashed line
- `wavy` - Displays the line as a wavy line

Specify the Thickness for the Decoration Line

The CSS [text-decoration-thickness](#) property is used to set the thickness of the decoration line.

```
<> data.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <style>
5  h1 {
6      text-decoration-line: underline;
7      text-decoration-thickness: auto; /* this is default */
8  }
9
10 h2 {
11     text-decoration-line: underline;
12     text-decoration-thickness: 5px;
13 }
14
15 h3 {
16     text-decoration-line: underline;
17     text-decoration-thickness: 25%;
18 }
19
20 p {
21     text-decoration-line: underline;
22     text-decoration-color: red;
23     text-decoration-style: double;
24     text-decoration-thickness: 5px;
25 }
26 </style>
27 </head>
28 <body>
29 <h1>Heading 1</h1>
30 <h2>Heading 2</h2>
31 <h3>Heading 3</h3>
32 <p>A paragraph.</p>
33 </body>
34 </html>
35
```

CSS Text Transformation

The CSS [text-transform](#) property is used to control the capitalization of text in an element.

It can be used to transform a text into uppercase or lowercase letters, or capitalize the first letter of each word, without changing the original content in HTML.

This property can have one of the following values:

- `none` - No transformation. Text renders as it is
- `capitalize` - Transforms the first character of each word to uppercase
- `uppercase` - Transforms all characters to uppercase
- `lowercase` - Transforms all characters to lowercase

```
<> data.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <style>
5  p.uppercase {
6    text-transform: uppercase;
7  }
8
9  p.lowercase {
10   text-transform: lowercase;
11 }
12
13 p.capitalize {
14   text-transform: capitalize;
15 }
16 </style>
17 </head>
18 <body>
19 <h1>Using the text-transform property</h1>
20
21 <p class="uppercase">This text is transformed to uppercase.</p>
22 <p class="lowercase">This text is transformed to lowercase.</p>
23 <p class="capitalize">This text is capitalized.</p>
24 </body>
25 </html>
26
```

CSS Text Spacing

CSS has several properties to control text spacing.

In this chapter you will learn about the following properties:

- [text-indent](#)
- [letter-spacing](#)
- [line-height](#)
- [word-spacing](#)
- [white-space](#)
-

The CSS [text-indent](#) property is used to specify the indentation of the first line in a text-block.

The CSS [letter-spacing](#) property is used to specify the space between the characters in a text.

The CSS [line-height](#) property is used to specify the space between lines.

The CSS [word-spacing](#) property is used to specify the space between the words in a text.

The CSS [white-space](#) property specifies how white-space inside an element is handled.

This property can have one of the following values:

- `normal`
- `nowrap`
- `pre`
- `pre-line`
- `pre-wrap`

```
<> data.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <style>
5        p {
6          white-space: nowrap;
7        }
8      </style>
9    </head>
10   <body>
11     <h1>Using white-space</h1>
12
13     <p>
14       This is some text that will not wrap. This is some text that will not
15       wrap. This is some text that will not wrap. This is some text that will
16       not wrap. This is some text that will not wrap. This is some text that
17       will not wrap. This is some text that will not wrap. This is some text
18       that will not wrap. This is some text that will not wrap.
19     </p>
20
21     <p>Try to remove the white-space property to see the difference!</p>
22   </body>
23 </html>
24
```

CSS Text Shadow

The [text-shadow](#) property adds shadow to text.

In its simplest use, you only specify the horizontal and the vertical shadow.

In addition, you can add a shadow color and blur effect.

```
<> data.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <style>
5        h1 {
6          text-shadow: 2px 2px 5px red;
7        }
8      </style>
9    </head>
10   <body>
11     <h1>Text-shadow effect!</h1>
12   </body>
13 </html>
14
```

CSS Fonts

The CSS font-family Property

The CSS [font-family](#) property specifies the font for an element.

Tip: The [font-family](#) property should hold several font names as a "fallback" system. If the browser does not support the first font, it tries the next font. The font names should be separated with a comma.

Always start with the font you want, and always end with a generic family, to let the browser pick a similar font in the generic family, if no other fonts are available.

CSS Font Style

The CSS [font-style](#) property specifies the font style for a text..

This property can have one of the following values:

- `normal` - The text is shown normally
- `italic` - The text is shown in italics
- `oblique` - The text is "leaning" (oblique is very similar to italic)

CSS Font Weight

The CSS [`font-weight`](#) property specifies how thick or thin characters in text should be displayed.

This property can have one of the following values:

- `normal` - This is default. Defines normal characters
- `bold` - Defines thick characters
- `bolder` - Defines thicker characters
- `lighter` - Defines lighter characters
- `100-900` - Defines from thin to thick characters. 400 is the same as normal, and 700 is the same as bold

CSS Font Variant

The CSS [`font-variant`](#) property specifies whether or not a text should be displayed in a small-caps font.

In a small-caps font, all lowercase letters are converted to uppercase letters. However, the converted uppercase letters appears in a smaller font size than the original uppercase letters in the text.

CSS Font Size

The CSS [`font-size`](#) property is used to specify the size of the text/font.

Being able to manage the text size is very important in web design.

However, you should not use font size adjustments to make paragraphs look like headings, or headings look like paragraphs. Always use the proper HTML tags, like `<h1>` - `<h6>` for headings and `<p>` for paragraphs.

The [`font-size`](#) can be set to an absolute size or to a relative size.

Absolute sizes:

- **px**: Pixels offers fixed and precise control over the font size.
- **xx-small, x-small, small, medium, large, x-large, xx-large**. These keywords has a predefined set of sizes in browsers.

CSS Icons

Bootstrap icons

```
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.13.1/font/bootstrap-icons.
min.css">
```

For eg

In your HTML you can add icons code given in bootstrap

```
<i class="bi bi-tiktok"></i>
```

CSS Links

```
<> data.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <style>
5      a:link,
6      a:visited {
7          background-color: blue;
8          color: white;
9          padding: 14px 25px;
10         text-align: center;
11         text-decoration: none;
12         display: inline-block;
13     }
14
15     a:hover,
16     a:active {
17         background-color: red;
18     }
19 </style>
20 </head>
21 <body>
22     <h2>Link Button</h2>
23
24     <p>A link styled as a button:</p>
25     <a href="default.asp" target="_blank">This is a link</a>
26 </body>
27 </html>
28
```

CSS Styling Lists

In HTML, there are two main types of lists:

- - unordered lists (list items are marked with bullets)
- - ordered lists (list items are marked with numbers or letters)

CSS has the following properties for styling HTML lists:

- [list-style-type](#) - Specifies the type of list-item marker
- [list-style-image](#) - Specifies an image as the list-item marker
- [list-style-position](#) - Specifies the position of the list-item markers
- [list-style](#) - A shorthand property for the properties above

```
ul.a {list-style-type: circle;}
ul.b {list-style-type: disc;}
ul.c {list-style-type: square;}

ol.d {list-style-type: upper-roman;}
ol.e {list-style-type: lower-roman;}
ol.f {list-style-type: lower-alpha;}
ol.g {list-style-type: decimal;}
```

CSS The display Property

The [display](#) property is an important CSS property for controlling layout. It specifies whether an HTML element is treated as a block or an inline element.

Every HTML element has a default display value, depending on what type of element it is. The default display value for most elements is `block` or `inline`.

The [display](#) property is used to change the default display behavior of HTML elements.

Block-level Elements

A block-level element ALWAYS starts on a new line and takes up the full width available (stretches out to the left and right as far as it can).

The `<div>` element is a block-level element.

Examples of block-level elements:

- `<div>`
- `<h1>` - `<h6>`
- `<p>`
- `<form>`
- `<header>`
- `<footer>`
- `<section>`

Inline Elements

An inline element DOES NOT start on a new line and only takes up as much width as necessary.

This is an inline element inside a paragraph.

Examples of inline elements:

- `<span>`
- `<a>`
- `<img>`

Common display Values

The CSS `display` property has many values. The following table lists the most commonly used:

Value	Description
inline	Displays an element as an inline element
block	Displays an element as a block element
contents	Makes the container disappear, making its child elements children of the element the next level up in the DOM
flex	Displays an element as a block-level flex container
grid	Displays an element as a block-level grid container
inline-block	Displays an element as an inline-level block container. The element itself is formatted as an inline element, but you can apply height, width, padding, and margin values
none	The element is completely hidden from the document flow (does not take up any space).

```
<> data.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <style>
5  p {
6      background-color: blue;
7      padding: 10px;
8  }
9
10 span {
11     background-color: red;
12     color: white;
13     padding: 10px;
14 }
15
16 p.ex1 {
17     display: none;
18 }
19 p.ex2 {
20     display: inline;
21 }
22 p.ex3 {
23     display: block;
24 }
25 p.ex4 {
26     display: inline-block;
27 }
28 p.ex5 {
29     display: flex;
30 }
31 p.ex6 {
32     display: grid;
33 }
34 </style>
35 </head>
```

Ln 82, Col 1 — Spaces: 2 — UTF-8 — CRLF — (1) HTML — @ Go Live

```
</head>
<body>
  <h1>The display Property</h1>

  <h2>display: none:</h2>
  <p class="ex1">
    <span>Text 1</span>
    <span>Text 2</span>
    <span>Text 3</span>
  </p>

  <h2>display: inline:</h2>
  <p class="ex2">
    <span>Text 1</span>
    <span>Text 2</span>
    <span>Text 3</span>
  </p>

  <h2>display: block:</h2>
  <p class="ex3">
    <span>Text 1</span>
    <span>Text 2</span>
    <span>Text 3</span>
  </p>

  <h2>display: inline-block:</h2>
  <p class="ex4">
    <span>Text 1</span>
    <span>Text 2</span>
    <span>Text 3</span>
  </p>
```

```
<> data.html > html > head
2   <html>
36  <body>
66
67    <h2>display: flex:</h2>
68    <p class="ex5">
69      <span>Text 1</span>
70      <span>Text 2</span>
71      <span>Text 3</span>
72    </p>
73
74    <h2>display: grid:</h2>
75    <p class="ex6">
76      <span>Text 1</span>
77      <span>Text 2</span>
78      <span>Text 3</span>
79    </p>
80  </body>
81 </html>
82
```

CSS The max-width Property

The `max-width` property defines the maximum width of an element.

This property prevents the element's width from being larger than the specified value (it can be smaller, but not larger).

This property is useful for creating responsive layouts and to ensure content readability across various screen sizes.

Problem with width:

Here we have a horizontally centered `<div>` element with a specific `width` (600px):

This `<div>` element has a width of 600px, and margin set to auto. This `<div>` element has a width of 600px, and margin set to auto.

What happens to the `<div>` above if the browser window is smaller than the width of the element? Some of the content will not show, and the browser might add a horizontal scrollbar to the page.

Using max-width instead:

Now, we use the `max-width` property instead. This will improve the browser's handling of small windows:

This <div> element has a max-width of 600px, and margin set to auto. This <div> element has a max-width of 600px, and margin set to auto.

Tip: Resize the browser window to less than 600px wide, to see the difference between the two <div>s!

Here is the CSS code for the two <div>s above:

CSS The position Property

CSS positioning is about controlling the placement of elements within a web page.

With CSS positioning, you can override the normal document flow.

The `position` property specifies the positioning type for an element.

This property can have one of the following values:

- `static` - This is default
- `relative`
- `fixed`
- `absolute`
- `sticky`

Elements are then positioned to their final location with the `top`, `bottom`, `left`, and `right` properties.

All HTML elements are positioned static by default.

Static positioned elements are not affected by the top, bottom, left, and right properties.

An element with `position: static;` is always positioned according to the normal flow of the page:

This <div> element has position: static;

Here is the CSS that is used:

An element with `position: relative;` is positioned relative to its normal position in the document flow.

Setting the top, right, bottom, and left properties will cause the element to be adjusted away from its normal position. Other content will not be adjusted to fit into any gap left by the element.

This <div> has position: relative, and is skewed 30 px to the right of its normal position

An element with `position: fixed;` is positioned relative to the viewport, which means it always stays in the same place even if the page is scrolled. The top, right, bottom, and left properties are used to set the final location of the element.

A fixed element does not leave a gap in the page where it would normally have been located.

Notice the fixed element in the lower-right corner of the page. Here is the CSS that is used:

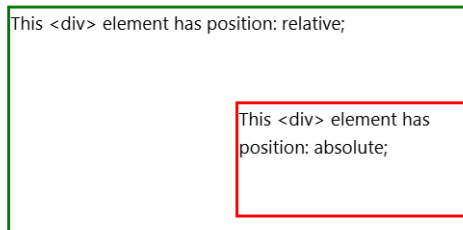
CSS position: absolute

An element with `position: absolute;` is positioned relative to the nearest positioned ancestor (with position other than static).

However, if an absolute positioned element has no positioned ancestors, it uses the document body, and moves along with page scrolling.

Note: Absolute positioned elements are removed from the normal document flow, and can overlap other elements.

Here is a simple example:



An element with `position: sticky;` toggles between a `relative` and `fixed` position, depending on the scroll position.

A sticky element is positioned relative until a certain scroll position is reached - then it "sticks" in that place (like `position: fixed`).

```

<!DOCTYPE html>
<html>
  <head>
    <style>
      div.relative {
        position: relative;
        width: 400px;
        height: 200px;
        border: 3px solid green;
      }

      div.absolute {
        position: absolute;
        top: 80px;
        right: 0;
        width: 200px;
        height: 100px;
        border: 3px solid red;
      }
    </style>
  </head>
  <body>
    <h2>Using position: absolute;</h2>

    <p>
      An element with position: absolute; is positioned relative to the nearest
      positioned ancestor:
    </p>

    <div class="relative">
      This div element has position: relative;
      <div class="absolute">This div element has position: absolute;</div>
    </div>
  </body>
</html>

```

CSS The z-index Property

The [z-index](#) property specifies the stack order of positioned elements.

The stack order defines which element should be placed in front or behind other elements.

When elements are positioned, they can overlap other elements.

An element can have a positive or negative stack order (z-index):

```

data.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <style>
5  .container {
6    position: relative;
7  }
8
9  .black-box {
10   position: relative;
11   z-index: 1;
12   border: 2px solid black;
13   height: 100px;
14   margin: 30px;
15 }
16
17 .gray-box {
18   position: absolute;
19   z-index: 3; /* gray box will be above both green and black box */
20   background: lightgray;
21   height: 60px;
22   width: 70%;
23   left: 50px;
24   top: 50px;
25 }
26
27 .green-box {
28   position: absolute;
29   z-index: 2; /* green box will be above black box */
30   background: lightgreen;
31   width: 35%;
32   left: 270px;
33   top: -15px;
34   height: 100px;
35 }
36 </style>
37 </head>
38 <body>
39 <h1>Z-index Example</h1>
40
41 <p>
42   An element with greater stack order is always above an element with a
43   lower stack order.
44 </p>
45
46 <div class="container">
47   <div class="black-box">Black box (z-index: 1)</div>
48   <div class="gray-box">Gray box (z-index: 3)</div>
49   <div class="green-box">Green box (z-index: 2)</div>
50 </div>
51 </body>
52 </html>

```

Note : Please check the difference by removing z index from the above code

CSS The overflow Property

The CSS [overflow](#) property controls what happens to content that is too big to fit into an area.

It specifies whether to clip the content or to add scrollbars when the content of an element is too big.

The [overflow](#) property has the following values:

- **visible** - Default. The overflow is not clipped. The content renders outside the element's box
- **hidden** - The overflow is clipped, and the rest of the content is hidden
- **scroll** - Scrollbars are added. User must scroll to see all content
- **auto** - Similar to **scroll**, but adds scrollbars only when necessary

CSS overflow-x and overflow-y

The [overflow-x](#) and [overflow-y](#) properties specifies whether to change the overflow of content just horizontally or vertically (or both):

- [overflow-x](#) specifies what to do with the left/right edges of the content.
- [overflow-y](#) specifies what to do with the top/bottom edges of the content.

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      div {
        background-color: coral;
        width: 200px;
        height: 100px;
        border: 1px solid black;
        overflow: scroll;
      }
    </style>
  </head>
  <body>
    <h2>Overflow: scroll</h2>

    <p>
      Setting the overflow value to scroll, the overflow is clipped and a
      scrollbar is added to scroll inside the box. Note that this will add a
      scrollbar both horizontally and vertically (even if you do not need it):
    </p>

    <div>
      You can use the overflow property when you want to have better control of
      the layout. The overflow property specifies what happens if content
      overflows an element's box.
    </div>
  </body>
</html>
```

CSS Float

The `float` property specifies how an element should float within its container.

It places an element on the left or right side of its container, allowing text and inline elements to wrap around it.

The image inside this `<div>` is floated to the right.

If you make the screen smaller, you will see that the text wraps around the image.



The CSS float Property

The `float` property is used for positioning and formatting content e.g. let an image float left to the text in a container.

This property can have one of the following values:

- `left` - The element floats to the left of its container
- `right` - The element floats to the right of its container
- `none` - Default. The element does not float and is displayed just where it occurs in the text
- `inherit` - The element inherits the float value of its parent

Tip: The `float` property is often used to wrap text around images!

CSS Float Property

Property	Description
<code>float</code>	Specifies whether an element should float to the left, right, or not at all

The CSS clear Property

The `clear` property specifies what should happen with the element that is NEXT to a floating element.

The `clear` property prevents elements from wrapping around or beside the floated content.

This property can have one of the following values:

- `none` - Default. Allows elements to float on either side

- `left` - The element is pushed below floated elements on its left side
- `right` - The element is pushed below floated elements, on its right side
- `both` - The element is pushed below floated elements, on both its left and right side
- `inherit` - The element inherits the clear value from its parent

CSS Flexbox

What is CSS Flexbox?

Flexbox is short for the Flexible Box Layout module.

Flexbox is a layout method for arranging items in rows or columns.

Flexbox makes it easier to design a flexible responsive layout structure, without using float or positioning.

CSS Flexible Box Layout Module

Before the Flexible Box Layout module, there were four layout modes:

- Block, for sections in a webpage
- Inline, for text
- Table, for two-dimensional table data
- Positioned, for explicit position of an element

CSS flexbox is supported in all modern browsers.

CSS Flexbox Components

A flexbox always consists of:

- a Flex Container - the parent (container) `<div>` element
- Flex Items - the items inside the container `<div>`

A Flex Container with Three Flex Items

To start using CSS Flexbox, you need to first define a flex container.

The flex container becomes flexible by setting the `display` property to `flex`.



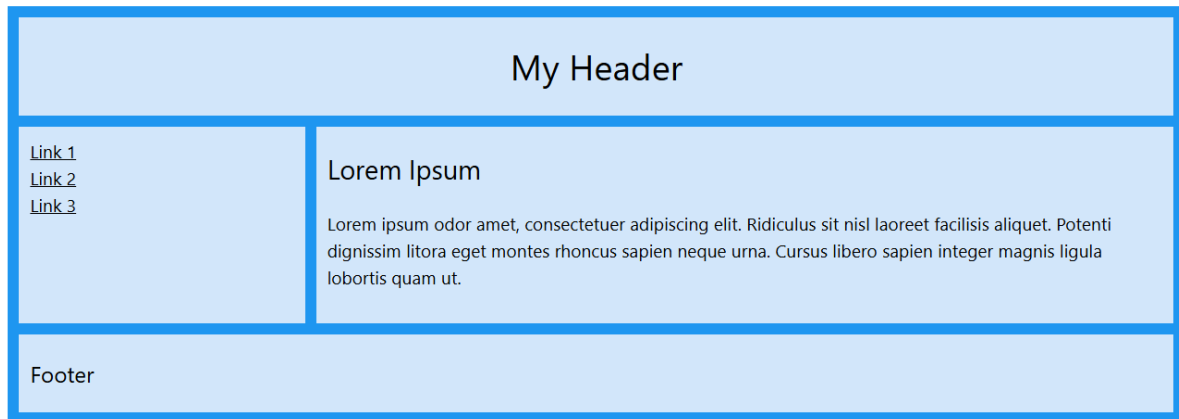
The element above represents a **flex container** (the blue area) with three **flex items**.

```
data.html > ...
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <style>
5  .flex-container {
6    display: flex;
7    background-color: #007bff;
8  }
9
10 .flex-container > div {
11   background-color: #f1f1f1;
12   margin: 10px;
13   padding: 20px;
14   font-size: 30px;
15 }
16 </style>
17 </head>
18 <body>
19   <h1>Create a Flex Container</h1>
20
21   <div class="flex-container">
22     <div>1</div>
23     <div>2</div>
24     <div>3</div>
25   </div>
26
27   <p>
28     A Flexible Layout must have a parent element with the
29     <em>display</em> property set to <em>flex</em>.
30   </p>
31
32   <p>
33     Direct child elements(s) of the flexible container automatically becomes
34     flexible items.
35   </p>
36 </body>
37 </html>
```

Note : Refer this URL for more understanding on flexbox

<https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

CSS Grid Layout Module



The Grid Layout Module offers a grid-based layout system, with rows and columns.

The Grid Layout Module allows developers to easily create complex web layouts.

The Grid Layout Module makes it easier to design a responsive layout structure, without using float or positioning.

The CSS grid properties are supported in all modern browsers.

Grid vs. Flexbox

The CSS Grid Layout should be used for two-dimensional layout, with rows AND columns.

The [CSS Flexbox Layout](#) should be used for one-dimensional layout, with rows OR columns.

CSS Grid Components

A grid always consists of:

- a **Grid Container** - the parent (container) <div> element
- **Grid Items** - the items inside the container <div>


```

1  <!DOCTYPE html>
2  <html>
3  <head>
4    <style>
5      .container {
6        display: grid;
7        grid-template-columns: auto auto auto;
8        background-color: #dodgerblue;
9        padding: 10px;
10     }
11     .container > div {
12       background-color: #f1f1f1;
13       border: 1px solid black;
14       padding: 10px;
15       font-size: 30px;
16       text-align: center;
17     }
18   </style>
19 </head>
20 <body>
21   <h1>Grid Layout</h1>
22
23   <p>
24     A Grid Layout must have a parent element with the
25     <em>display</em> property set to <em>grid</em> or <em>inline-grid</em>.
26   </p>
27
28   <p>
29     Direct child element(s) of the grid container automatically becomes grid
30     items.
31   </p>
32
33   <div class="container">
34     <div>1</div>
35     <div>2</div>
36     <div>3</div>
37     <div>4</div>
38     <div>5</div>
39     <div>6</div>
40     <div>7</div>
41     <div>8</div>
42   </div>
43 </body>

```

Note: Please refer this URL for more information

<https://css-tricks.com/snippets/css/complete-guide-grid/>

CSS Gradients

```

.class-name {
  background-image: linear-gradient(■red, ■yellow);
  background-image: linear-gradient(to right, ■red, ■yellow);
  background-image: linear-gradient(
    to right,
    ■red,
    ■orange,
    ■yellow,
    ■green,
    ■blue,
    ■indigo,
    ■violet
  );
  background-image: linear-gradient(
    to right,
    ■rgba(255, 0, 0, 0),
    ■rgba(255, 0, 0, 1)
  );
  background-image: repeating-linear-gradient(■red, ■yellow 10%, ■green 20%);
  background-image: repeating-radial-gradient(■red, ■yellow 10%, ■green 15%);
  background-image: conic-gradient(■red, ■yellow, ■green, ■blue, ■black);
}

```

CSS Animations

CSS allows animation of HTML elements without using JavaScript!

What are CSS Animations?

An animation lets an element gradually change from one style to another.

You can change as many CSS properties you want, as many times as you want.

To use CSS animation, you must first specify some keyframes for the animation.

Keyframes hold what styles the element will have at certain times.

The CSS @keyframes Rule

When you specify CSS styles inside the [@keyframes](#) rule, the animation will gradually change from the current style to the new style at certain times.

To get an animation to work, you must bind the animation to an element.

The following example binds the "example" animation to the <div> element. The animation will last for 4 seconds, and it will gradually change the background-color of the <div> element from "red" to "yellow":

```
<html>
<head>
  <style>
    div {
      width: 100px;
      height: 100px;
      background-color: red;
      position: relative;
      animation-name: example;
      animation-duration: 4s;
    }

    @keyframes example {
      0% {
        background-color: red;
        left: 0px;
        top: 0px;
      }
      25% {
        background-color: yellow;
        left: 200px;
        top: 0px;
      }
      50% {
        background-color: blue;
        left: 200px;
        top: 200px;
      }
      75% {
        background-color: green;
        left: 0px;
        top: 200px;
      }
      100% {
        background-color: red;
        left: 0px;
        top: 0px;
      }
    }
  </style>
</head>
<body>
  <h1>CSS Animation</h1>

  <div></div>

  <p>
    <b>Note:</b> When an animation is finished, it goes back to its original
    style.
  </p>
</body>
</html>
```

```

<!DOCTYPE html>
<html>
  <head>
    <style>
      div {
        width: 100px;
        height: 50px;
        background-color: red;
        font-weight: bold;
        position: relative;
        animation: mymove 5s;
        animation-fill-mode: forwards;
      }

      #div1 {
        animation-timing-function: linear;
      }
      #div2 {
        animation-timing-function: ease;
      }
      #div3 {
        animation-timing-function: ease-in;
      }
      #div4 {
        animation-timing-function: ease-out;
      }
      #div5 {
        animation-timing-function: ease-in-out;
      }

      @keyframes mymove {
        from {
          left: 0px;
        }
        to {
          left: 300px;
        }
      }
    </style>
  </head>
  <body>
    <h1>CSS Animation</h1>

    <p>
      The animation-timing-function property specifies the speed curve of the
      animation. The following example shows some of the different speed curves
      that can be used:
    </p>

    <div id="div1">linear</div>
    <div id="div2">ease</div>
    <div id="div3">ease-in</div>
    <div id="div4">ease-out</div>
    <div id="div5">ease-in-out</div>
  </body>
</html>

```

Property	Description
@keyframes	Specifies the animation code
animation	A shorthand property for setting all the animation properties
animation-delay	Specifies a delay for the start of an animation
animation-direction	Specifies whether an animation should be played forwards, backwards or in alternate cycles
animation-duration	Specifies how long time an animation should take to complete one cycle
animation-fill-mode	Specifies a style for the element when the animation is not playing (before it starts, after it ends, or both)
animation-iteration-count	Specifies the number of times an animation should be played
animation-name	Specifies the name of the @keyframes animation
animation-play-state	Specifies whether the animation is running or paused
animation-timing-function	Specifies the speed curve of the animation

CSS Revision made Easy

Please go through the image below to recap the commonly used attributes in CSS.

```
background-color: DodgerBlue;
color: Tomato;
background-color: rgb(255, 99, 71);
background-color: #ff6347;
background-color: hsl(9, 100%, 64%);
opacity: 0.3;
background-image: url("paper.gif");
background-repeat: repeat-x;
background-position: right top;
background-attachment: fixed;
border-style: solid;
border-width: 5px;
border-color: red;
border: 5px solid red;
margin-top: 100px;
margin-bottom: 100px;
margin-right: 150px;
margin-left: 80px;
margin: 25px;
margin: 25px 50px 75px 100px;
padding-top: 50px;
padding-right: 30px;
padding-bottom: 50px;
padding-left: 80px;
padding: 25px 50px 75px 100px;
padding: 25px;
height: 200px;
width: 50%;
outline-style: solid;
outline-width: 8px;
outline-color: green;
text-transform: uppercase;
font-family: "Times New Roman", Times, serif;
```

```
font-style: normal;
font-size: 40px;
list-style-type: circle;
list-style-type: upper-roman;
border-collapse: collapse; /* use this to remove unnecessary borders in Table */
display: inline;
display: block;
display: inline-block;
max-width: 500px;
position: static;
position: relative;
position: fixed;
position: absolute;
position: sticky;
z-index: -1;
overflow: visible;
float: right;
text-align: center;
border-radius: 25px;
background-image: linear-gradient(■red, ■yellow);
background-image: linear-gradient(to bottom right, ■red, ■yellow);
animation-name: example;
animation-duration: 4s;
```

